Menyajikan Output (HTML & Data)

Pemrograman Web • 11 slides • Generated on 11 October 2026, 22:42

Menyajikan Output: HTML & JSON

- **Materi:**
  - `html/template` — HTML pindah ke file sendiri, escape otomatis
  - Mengirim data, `{{range}}`, `{{if}}`, fungsi template
  - Layout reusable: header, footer, konten
  - File statis: CSS, JS, gambar

Mengenal `html/template`

Slide 2

- Template = file HTML + placeholder `{{...}}` yang diisi data
- Langkah:

```
import "html/template"

tpl, err := template.ParseFiles("templates/profil.html")   // 1. parse file
if err != nil { ... }
err = tpl.Execute(w, data)                                  // 2. isi & tulis ke w
```

- `template.Must` , agar kalau ada parsing error, akan gagal build (memicu panic):

```
var tplProfil = template.Must(template.ParseFiles("templates/profil.html"))
```

- Buat template sebagai variabel package-level + `Must` → parse saat program nyala

Pola Dasar: File Template + Handler

Slide 3

- `templates/profil.html`:

```
<!DOCTYPE html>
<html lang="id">
<head><meta charset="utf-8"><title>Profil</title></head>
<body>
  <h1>{{.Nama}}</h1>
  <p>Email: {{.Email}}</p>
</body>
</html>
```

- `main.go`:

```
type Profil struct {
    Nama  string
    Email string
}

var tplProfil = template.Must(template.ParseFiles("templates/profil.html"))
```

```
func profilHandler(w http.ResponseWriter, r *http.Request) {
    data := Profil{Nama: "Anggy", Email: "anggy@mail.com"}
    if err := tplProfil.Execute(w, data); err != nil {
        log.Printf("render profil gagal: %v", err)
    }
}
```

- `.` (dot) = data yang dikirim ke `Execute`. `{{.Nama}}` = field `Nama` dari data
- Field **harus diekspor** (huruf besar) agar bisa diakses di template
- Path `templates/...` relatif ke folder tempat `go run` dijalankan

Contextual Escaping

Slide 4

- `html/template` melihat **posisi** placeholder, lalu memilih escape yang tepat:

```
<p>{{.X}}</p>            <!-- <b>hi</b>          ?  <b>hi</b>  -->
<p title="{{.X}}">       <!-- "q"                ?  "q"            -->
<a href="{{.X}}">        <!-- javascript:alert(1) ?  #ZgotmplZ (dibuang)   -->
<script>var s = {{.X}};  <!-- jadi string JSON yang valid                   -->
```

- Escape dilakukan sesuai konteks, secara otomatis
- Kalau mau render apa adanya, pakai `template.HTML`

```
data.Isi = template.HTML("<b>tebal</b>")   // ditampilkan apa adanya
```

- Hanya untuk HTML yang **kita buat sendiri**, bukan dari input user

Data untuk Template

Slide 5

- `.` bisa berupa struct, map, slice, apa pun yang dikirim ke `Execute`
- **Struct per halaman** — cara yang dianjurkan:

```
type halamanDaftar struct {
    Title  string
    Produk []Produk
}

tplDaftar.Execute(w, halamanDaftar{
    Title:  "Daftar Produk",
    Produk: daftarProduk,
})
```

Perulangan: `{{range}}`

Slide 6

```
<ul>
  {{range .Produk}}
  <li>{{.Nama}} — {{.Harga}}</li>
  {{else}}
  <li><em>Belum ada produk.</em></li>
  {{end}}
</ul>
```

- Di dalam `range`, `.` **berganti** menjadi elemen yang sedang diproses
- `{{else}}` jalan kalau slice **kosong atau nil**
- Butuh data induk dari dalam loop? `$` selalu menunjuk ke data awal:

```
{{range .Produk}}
  <li>{{.Nama}} ({{$.Title}})</li>
{{end}}
```

- Kita juga bisa akses key dalam slice:

```
{{range $i, $p := .Produk}}
  <li>{{$i}}. {{$p.Nama}}</li>
{{end}}
```

Kondisi: `{{if}}` dan `{{with}}`

Slide 7

```
{{if gt .Stok 0}}
  <td>{{.Stok}}</td>
{{else}}
  <td><em>habis</em></td>
{{end}}
```

- `{{if .X}}` bernilai salah untuk **zero value**: `0`, `""`, `nil`, `false`, slice kosong
- Perbandingan lewat fungsi, bukan operator: `eq`, `ne`, `lt`, `le`, `gt`, `ge`
- Gabungan: `and`, `or`, `not`

```
{{if and .LoggedIn (eq .Role "admin")}}<a href="/admin">Admin</a>{{end}}
```

- `{{with}}` = `if` + ganti `.` ke nilai itu, kalau tidak kosong:

```
{{with .User}}
  <p>Halo, {{.Nama}}</p>        <!-- di sini . = User -->
{{else}}
```

```
  <a href="/login">Login</a>
{{end}}
```

Fungsi di Template

Slide 8

- Bawaan yang sering dipakai: `len`, `printf`, `index`

```
<p>{{len .Produk}} produk</p>
<p>Rating: {{printf "%.1f" .Rating}}</p>
```

- Butuh format sendiri, mis. `12000000` → `Rp 12.000.000`? Daftarkan fungsi Go:

```
func rupiah(n int) string { ... }

var funcs = template.FuncMap{"rupiah": rupiah}

var tpl = template.Must(
    template.New("layout").Funcs(funcs).ParseFiles("templates/layout.html", ...),
)
```

```
<td>{{rupiah .Harga}}</td>
```

Layout Reusable

Slide 9

- Header, footer, `<head>` sama di semua halaman → tulis **sekali**

- `templates/layout.html`:

```
{{define "layout"}}
<!DOCTYPE html>
<html lang="id">
<head><meta charset="utf-8"><title>{{.Title}} — Toko</title></head>
<body>
  {{template "header" .}}
  <main>{{template "content" .}}</main>
  {{template "footer" .}}
</body>
</html>
{{end}}
```

- `templates/header.html`:

```
{{define "header"}}
<header><h1>Toko</h1></header>
{{end}}
```

- `templates/produk-list.html`:

```
{{define "content"}}
<h2>{{.Title}}</h2>
...
{{end}}
```

- `{{define "nama"}}` memberi nama potongan, `{{template "nama" .}}` memanggilnya
- Titik di `{{template "header" .}}` **wajib** — tanpa itu `.` di dalamnya `nil`

Template per Halaman

Slide 10

```
func parseHalaman(file string) *template.Template {
    return template.Must(template.New("layout").Funcs(funcs).ParseFiles(
        "templates/layout.html",
        "templates/header.html",
        "templates/footer.html",
        "templates/"+file,
    ))
}

var (
    tplDaftar = parseHalaman("produk-list.html")
    tplDetail = parseHalaman("produk-detail.html")
)
```

- Render lewat **nama** template, bukan `Execute`:

```
tplDaftar.ExecuteTemplate(w, "layout", data)
```

File Statis: CSS, JS, Gambar

Slide 11

- File yang dikirim **apa adanya** — tidak lewat template

```
katalog/
??? main.go
??? templates/
??? public/          ? hanya yang boleh dilihat publik
    ??? style.css
    ??? logo.png
```

```
fs := http.FileServer(http.Dir("public"))
mux.Handle("GET /static/", http.StripPrefix("/static/", fs))
```

- Di template: `<link rel="stylesheet" href="/static/style.css">`

Catatan

- Tanpa `index.html`, `/static/` menampilkan **daftar isi folder**. Solusi: buat file `index.html` kosong
- Prefix `/static/` pastikan tidak bentrok dengan route halaman